

Human Activity Recognition Using Hidden Markov Models

Sharif Kiviiri - Formative 2, ML Techniques II

Github repository: https://github.com/Sharif2138/Formative-2_Hidden-Markov-Models

1. Background and Motivation

Wearable fitness trackers are useful for monitoring human activity, but they are not affordable or accessible everywhere, especially in low-resource settings where a smartphone is often the only sensor-equipped device a person owns. This project set out to explore whether a regular smartphone's built-in accelerometer and gyroscope, without any extra hardware, could be enough to reliably tell apart basic daily activities. This kind of low-cost activity recognition could eventually support things like elderly fall-risk monitoring, general health and fitness tracking, and rehabilitation progress tracking in places where dedicated wearable devices aren't practical. Since the true activity a person is doing at any moment can't be observed directly from raw sensor numbers, it was modeled as a Hidden Markov Model (HMM), where the activity is the hidden state and the sensor-derived features are the observations.

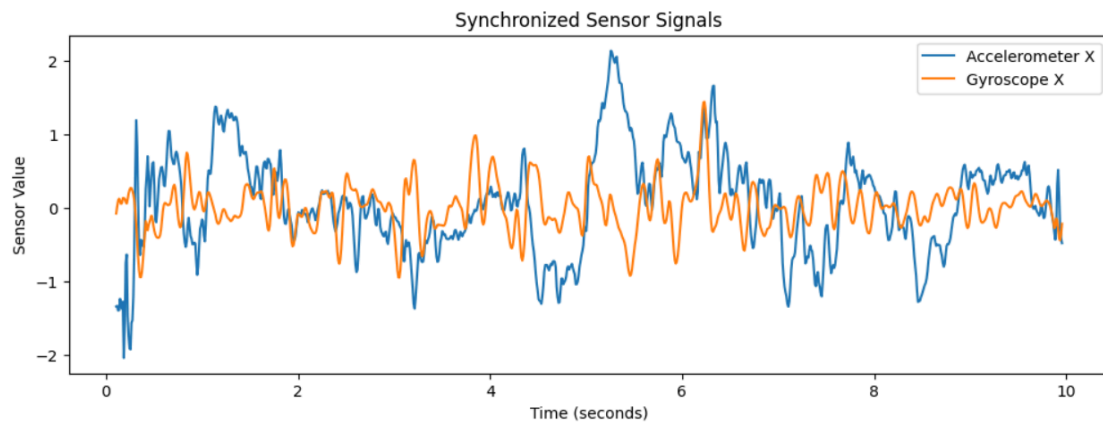
2. Data Collection and Preprocessing

Four activities were recorded, standing, walking, jumping, and still, using the Sensor Logger app, capturing accelerometer (x, y, z) and gyroscope (x, y, z) data for each session. In total, 50 recordings were collected across the four activities, roughly 12–13 recordings per activity, each lasting between 8 and 10 seconds, giving well over 1 minute 30 seconds of data per activity once combined.

Since a single device was used for all recordings (iphone 11), there was no need to reconcile differences between multiple phones, the effective sampling rate was calculated directly from the data itself, using two independent methods as a cross-check (total duration divided by number of samples, and the average gap between consecutive timestamps), and both agreed closely at around 100.7–100.8 Hz.

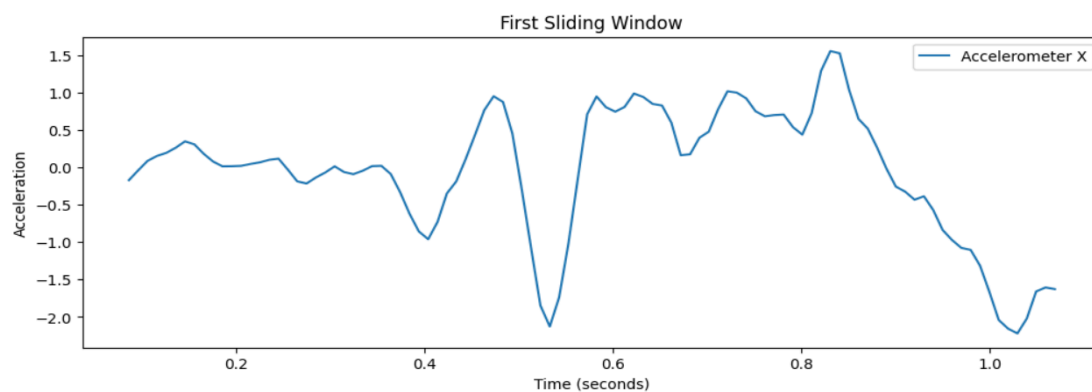
Synchronization. The accelerometer and gyroscope are separate sensors and don't always log at the exact same timestamps. This was checked across all 50 recordings, and only one recording was found where the two sensors had a mismatched number of rows (off by one). Because of this, accelerometer and gyroscope readings for each recording were merged using `pandas.merge_asof` with `direction="nearest"`, which pairs each accelerometer row with the closest gyroscope timestamp rather than requiring an exact match, this way no data is lost even if a recording has a small timing gap. `Pandas.merge` would work too because not tough misalignment existed in our data but `pandas.merge_of` is way safer.

Figure below shows how the two signals follow the same timeline after synchronization



Windowing. Rather than treating each raw sensor reading as a separate observation (which would be far too noisy and short to represent an activity), the synchronized signal was grouped into 1-second windows with 50% overlap. A 1-second window was chosen because, at the measured sampling rate of about 100 Hz, this gives roughly 100 samples per window, long enough to capture at least one full cycle of a repeating movement like a walking stride or a jump, but still short enough to react quickly if the activity changes. The 50% overlap (50-sample step size) means each window shares half its data with the next one, which gives more training windows from the same amount of recorded data without changing what a "window" physically represents. Applying this to all 50 recordings gave 924 total windows, with each recording contributing between 17 and 21 windows depending on its exact length.

The figure below shows one window extracted from the synchronized sensor signal



Feature extraction. For each window, both time-domain and frequency-domain features were computed per sensor axis (x/y/z for both accelerometer and gyroscope):

- **Time-domain:** mean, variance, standard deviation, RMS (root mean square), signal magnitude area (SMA) combining all three accelerometer axes, and correlation between axis pairs. These describe how much a signal moves and in what combined pattern, for example, jumping should have a much higher variance and RMS than standing or still, since there's far more physical movement involved.

- **Frequency-domain:** dominant frequency, spectral energy, and the first five FFT components, computed using `scipy.fft.rfft`. Before running the FFT, each signal's mean was subtracted first (removing the "DC" component), since otherwise the dominant frequency calculation kept picking up each signal's constant offset rather than its actual repeating motion, walking and jumping both have a distinct rhythm that only shows up once the offset is removed.

This gave 74 numeric features per window across all six sensor axes. Since some features are naturally on very different scales (variance can be in the tens, while correlation is bounded between -1 and 1), every feature was standardized (zero mean, unit variance) using `StandardScaler`. Importantly, the scaler was fit only on the training recordings and applied unchanged to the test recordings, so no information from the unseen test data leaked into how the features were scaled.

Train/test split. The data was split by whole recording session rather than by individual window, reserving at least two recordings per activity for testing (10 recordings total, covering all four activities), with the remaining 40 recordings used for training. Splitting by recording rather than by row matters because the windows overlap by 50%. Splitting individual windows randomly could let two overlapping windows from the same recording end up on both sides of the split, which would let the model see data during training that's nearly identical to what it's supposedly being tested on later.

3. HMM Setup and Implementation

The HMM structure follows the four required components: the hidden states (Z) represent the four activities; observations (X) are the 74-dimensional feature vectors per window; the transition matrix (A) describes how likely the model is to move from one activity to another between consecutive windows; and the emission model (B) describes what kind of feature values each hidden state tends to produce, modeled as a Gaussian distribution per state (`covariance_type="diag"`, meaning each feature's variance is modeled independently rather than trying to model every pairwise interaction, which this dataset's size can't support well).

The `hmmlearn` library's `GaussianHMM` was used, trained with the Baum–Welch algorithm (`.fit()`), and decoded with the Viterbi algorithm (`.predict()`). Training used a convergence check based on log-likelihood tolerance (`tol=1e-4`) rather than just running a fixed number of iterations, so training stops once the model actually stops improving. Since Baum–Welch is unsupervised, the model doesn't know which hidden state number corresponds to which real activity. After training, each hidden state was mapped to whichever activity label was most common among the training windows assigned to that state (majority vote).

This was run in two stages.

Experiment 1: baseline 4-state model. The first assumption was that each hidden state should correspond to exactly one activity, so a 4-state HMM was trained with a fixed random seed.

Training converged cleanly, with the log-likelihood climbing from about -62,396 to a final value of 22,682.17 over 13 iterations.

Experiment 2: state/seed search. As explained in the results section below, Experiment 1 didn't hold up. Two states ended up representing the same activity(jumping) while another activity(standing) got no state at all. All its predictions were under another similar activity (still). To fix this, a search was run over different numbers of hidden states (4, 5, and 6) combined with five different random seeds, retraining the model 15 times and checking, for each run, how many of the four real activities ended up with at least one dedicated state, then selecting the best-performing configuration among those. This led to a 6-state model (seed 0), which converged to a final log-likelihood of 41,102.43.

4. Results and Interpretation

Experiment 1 (4 states):

Activity	Number of Samples	Sensitivity	Specificity	Overall Accuracy
Jumping	36	1	1	0.686
Standing	59	0	1	0.686
Still	55	1	0.556	0.686
Walking	38	1	1	0.686

Jumping and walking were classified perfectly. However, standing was never predicted at all , every single "standing" test window was predicted as "still" instead. Looking at the state-to-activity mapping, the reason became clear: two of the four hidden states had both been assigned to "jumping," meaning no state was left to represent "standing" on its own. The model had used its limited number of states to separate two sub-patterns of jumping rather than separating standing from still.

To check whether this was a data problem (standing and still genuinely look the same to the sensors) or a model problem (the information exists but the model didn't use it), the raw per-axis acceleration means for standing versus still recordings were compared using boxplots. The `z_acc_mean` distributions were almost identical for both activities, but `y_acc_mean` and, to a lesser extent, `x_acc_mean` showed a real, visible separation. Standing recordings had a wider spread with a lower median (small natural body sway while standing upright), while still recordings clustered tightly around zero (phone resting flat and motionless). This confirmed the separating signal was present in the features; the 4-state model simply hadn't been given enough states to use it.

Experiment 2 (6 states, after the fix):

Activity	Number of Samples	Sensitivity	Specificity	Overall Accuracy
Jumping	36	1	1	0.984
Standing	59	1	0.977	0.984
Still	55	0.945	1	0.984
Walking	38	1	1	0.984

Once the model was allowed 6 hidden states instead of 4, every activity was represented: state 3 mapped to standing (176 training windows), state 1 to walking (194), state 2 to still (183), and jumping was split across three states (0, 4, and 5, with 55, 97, and 31 training windows respectively). This makes physical sense because jumping is naturally the most variable activity in the recordings (different jump heights and paces across sessions), so it's reasonable that the model found it needed more than one Gaussian cluster to describe it well, while standing, walking, and still were each consistent enough to be captured by a single state. Overall test accuracy improved greatly to 0.98.

Figure below shows transition matrix heatmap for Exp 2,

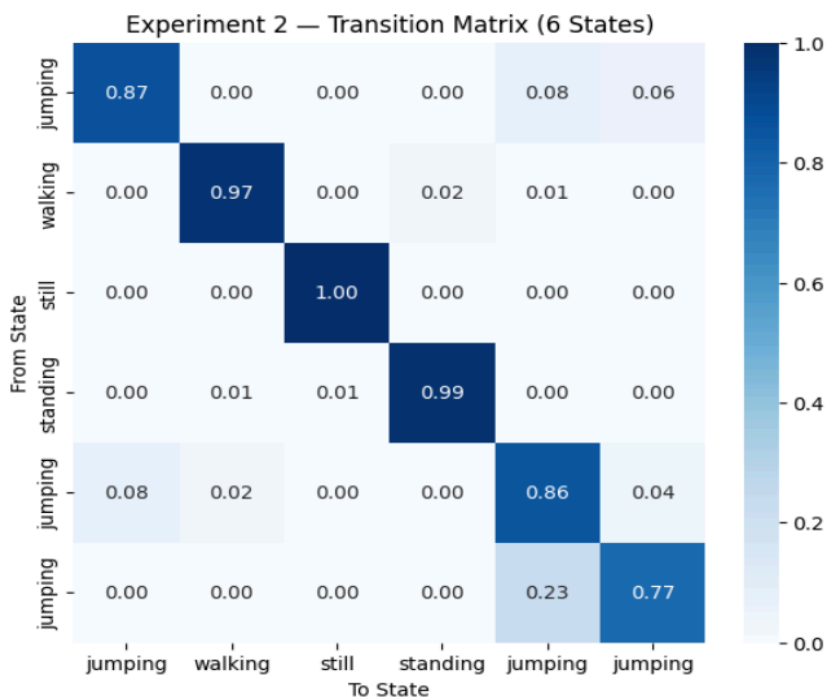


Figure below shows Emission Means per Hidden State for Exp 2

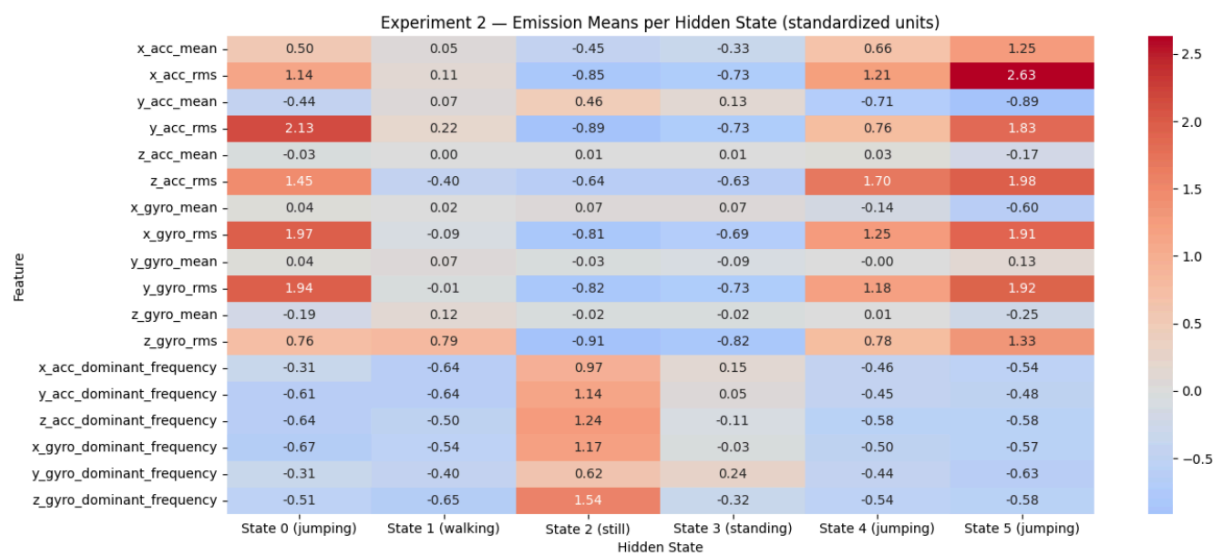


Figure below shows confusion matrix heatmap for Exp 2

Experiment 2 — Confusion Matrix (Unseen Test Recordings)

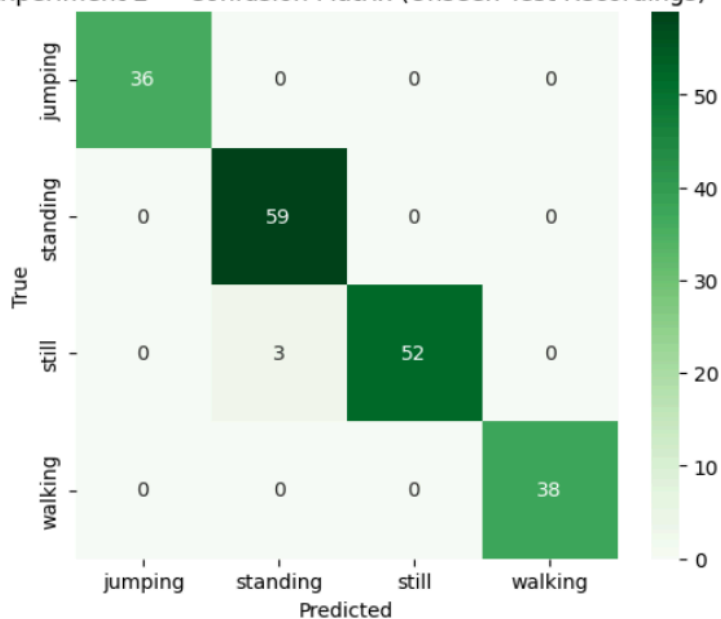
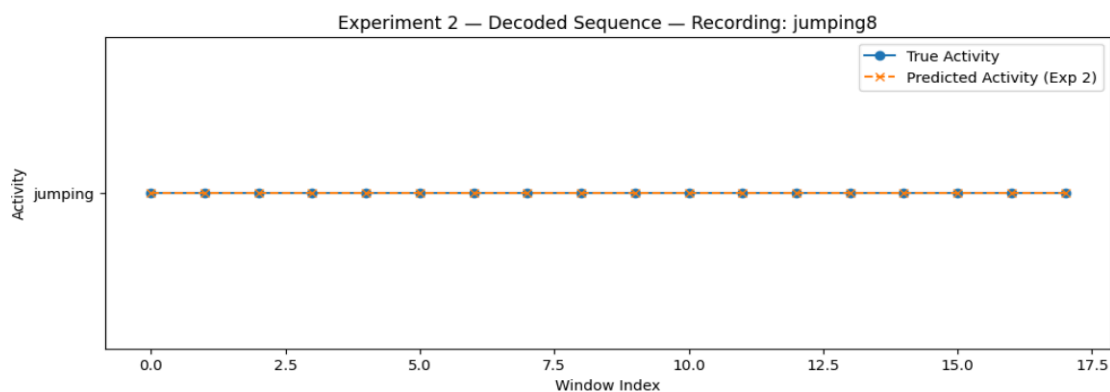


Figure below shows decoded sequence plot for Experiment 2



5. Discussion and Conclusion

Walking and jumping were consistently the easiest to classify correctly in both experiments, most likely because they involve large, rhythmic movement that is very different from the other two activities in both variance and dominant frequency. Standing and still were by far the hardest to separate, since both involve very little motion. The main distinguishing signal between them (small postural sway while standing versus a phone lying completely flat) is subtle and only shows up clearly on one or two axes rather than across the whole feature set.

Transition probabilities and realistic behavior. An activity would be expected to persist over consecutive one-second windows far more often than it changes i.e., a person doesn't switch from standing to jumping in an instant, back and forth. This should show up as the transition matrix's diagonal values being much larger than the off-diagonal ones. Within the 6-state model, the three jumping-related states would also be expected to transition heavily among themselves before eventually transitioning out into a different activity, since a person doesn't stop mid-jump.

Effect of sensor noise and sampling rate. Because the sampling rate directly determined the window size (1 second at ~100 Hz worked out to roughly 100 samples per window), any inconsistency in sampling rate would have shifted how much real time each window actually represented, which is part of why the sampling rate was checked using two independent methods rather than assuming a fixed number of samples equaled one second throughout. Minor sensor noise (small jitter in accelerometer readings even while still) is also part of why standing and still overlapped so much in the features. A completely noise-free sensor might have shown a cleaner separation.

Possible improvements. With more time, more data could be collected across more recording sessions and possibly different environments, to check whether the model generalizes beyond this particular recording setup. A feature specifically designed to capture phone orientation (for example, which axis carries the largest gravity-aligned component) could also be added, since this seemed to be the main signal separating standing from still and could be made more explicit rather than relying on the general per-axis means to capture it indirectly. Finally, testing with additional sensors like a barometer (which could pick up the small altitude changes during a jump) could help further separate the jumping sub-patterns found here rather than needing extra hidden states to represent them.

Overall, this project showed that building a working HMM isn't just about getting code to run. The first version of the model looked broken (missing an entire activity) even though the underlying code was correct, and the real fix came from questioning the assumption that the number of hidden states should exactly match the number of real-world activities.